Sprint 4 · Reto «Herramientas internas» · Entrega y rúbrica

Qué se sube y cómo se evalúa. Se lee **una vez**, el miércoles, antes de la entrega (misión 5), y se vuelve a la rúbrica cuando el equipo revisa su trabajo. Mientras programáis no hace falta: para eso están las tarjetas de misión.

- Volver al README del sprint · ficha del reto, para consultar.
- Vuestra hoja personal: autoevaluación y coevaluación.
- Cómo subir a GitHub paso a paso, con capturas: Cómo entregar en GitHub.

1 · Tabla A · Los ficheros: qué se sube

En total, al final del sprint se suben a `sprint-04/`: 4 ficheros del equipo (`kpis.py`, `test_kpis.py`, `README.md`, `salida.json`) **+ 1 autoevaluación y coevaluación por persona**, es decir, **4 + tantos ficheros como personas haya en el equipo** (un equipo de 3 sube 7). «El reto» no es un fichero aparte: son esos cuatro ficheros. Además, el CSV va a `data/` si aún no está, y los extras solo si los hacéis. La *review* y la *retro* se hacen de palabra: no se suben.

Todo lo que se sube va al repositorio privado del equipo (`ifcd107-equipo-…`), en la carpeta **`sprint-04/`**, salvo el CSV.

Fichero	Origen	Qué hacéis vosotros	¿Se sube?
`kpis.py`	**Lo modificáis.** Os damos una plantilla con las funciones vacías (`calcular_kpis` y la estructura de `main` ya vienen hechas)	Escribís el código de las funciones sobre la propia plantilla	**Sí** · `sprint-04/`
`test_kpis.py`	**Ya hecho.** Pruebas completas, que se ejecutan por fases (`python test_kpis.py 1`, 2 o sin número)	Nada: se copia tal cual y **no se modifica**	**Sí**, sin cambios · `sprint-04/`
`README.md`	**Lo creáis a partir de una plantilla.** Os damos `README_PLANTILLA.md` con huecos	Copiarla como `README.md` y rellenarla	**Sí** · `sprint-04/`
`salida.json`	**Lo genera el programa** al ejecutarlo con el CSV real	Ejecutar el programa; no se escribe a mano	**Sí** · `sprint-04/`
`reservas_turisdata.csv`	**Ya hecho.** Los datos reales (sintéticos)	Nada: se copia junto a `kpis.py` sin cambiarlo	**Sí**, sin cambios · `data/` (si aún no está)
Autoevaluación y coevaluación (`autoevaluacion_nombre.md`)	**Ya hecha, en blanco**	Cada persona copia la plantilla con su nombre y la rellena; la coevaluación va **dentro del mismo fichero**	**Sí**, **una por persona** (no una por equipo) · `sprint-04/`
`kpis_pandas.py` (extra)	**Lo modificáis.** Parte de `reto_herramientas/extra/plantilla_kpis_pandas.py`	Solo si el equipo llega al nivel extra	Solo si lo hacéis · `sprint-04/`
Pruebas propias (extra)	**Lo creáis vosotros**, en un fichero aparte	Solo si el equipo llega al nivel extra	Solo si lo hacéis · `sprint-04/`
`demo_cliente.py` y `reservas_demo.py`	**Ya hechos.** La demostración del cliente y sus 8 reservas de ejemplo	Ejecutarlos y leerlos (misión 0)	**No se suben**
`mision_1_datos.py` y `mision_2_decidir.py`	**Lo modificáis.** Ejercicios con huecos (`TODO`) y comprobación automática	Rellenar cada `TODO` (misiones 1 y 2)	**No se suben**
Copias de prueba de la misión 5 (`sin_col.csv`…)	**Las creáis** para romper el programa a propósito	Usarlas y borrarlas	**No se suben**

2 · Tabla B · Los requisitos: qué se evalúa

Qué tiene que cumplir lo que entregáis y hasta dónde llegar (mínimo, estándar, extra). **Es lo que se evalúa.**

Requisito	Qué se pide exactamente	Nivel	Dónde se comprueba
Leer el CSV	`leer_reservas`: convierte los tipos y **descarta y cuenta** las filas defectuosas (noches < 1, precio ≤ 0, `cancelada` distinto de 0/1, datos que no se pueden convertir)	Mínimo	`kpis.py`
Tasa de cancelación por canal	canceladas ÷ reservas del canal × 100, con 1 decimal	Mínimo	`kpis.py` · `tasa_cancelacion_por_canal`
Ingreso medio por reserva	media de `noches × precio_noche` de las reservas **no canceladas**, con 2 decimales (`0.0` si no hay)	Mínimo	`kpis.py` · `ingreso_medio_por_reserva`
Ocupación (proxy)	noches no canceladas ÷ (días del periodo × habitaciones) × 100; el periodo va de la primera a la última llegada. En el mínimo, con ayuda del docente	Estándar	`kpis.py` · `ocupacion_proxy`
Guardar el resultado	`exportar_json` y `main`: JSON en UTF-8 con los tres KPI y la sección `calidad` (filas válidas, descartadas y avisos). En el mínimo, con ayuda del docente	Estándar	`kpis.py` → `salida.json`
Avisar de los errores	Fichero inexistente o columna que falta → mensaje claro y código de salida 1, sin `Traceback`	Mínimo	`kpis.py` · `main` y `leer_reservas`
Resultado con el CSV real	`python kpis.py reservas_turisdata.csv salida.json --capacidad 40`	Mínimo	`salida.json`
Pruebas	`python test_kpis.py` pasa todo. **No se modifican** las dadas; las vuestras, aparte	Estándar	`test_kpis.py`
Código limpio	Docstring en cada función y estilo PEP 8	Estándar	`kpis.py`
README	Cómo se ejecuta y se prueba · qué es cada KPI · por qué la ocupación es un proxy · uso de la IA (qué pedisteis y qué revisasteis) · la revisión con la rúbrica. En el mínimo, versión corta	Mínimo	`README.md`
Git	Al menos 3 *commits* con mensaje claro, de personas distintas, sin claves ni datos personales	Mínimo	Historial del repositorio del equipo
Revisión y autoevaluación	Alguien que **no escribió esa parte** revisa el código y el README con la rúbrica (apartado 3) y anota nivel y prueba en el README; cada persona sube su hoja de autoevaluación y coevaluación	Estándar	`README.md` (sección «Revisión con la rúbrica») y `sprint-04/`
Extras	Pruebas propias · `kpis_pandas.py` con los mismos resultados · argumento `--por-mes` · `@dataclass` para `Reserva`	Extra	`test_kpis.py` · `kpis_pandas.py` · `kpis.py`

3 · Rúbrica del reto: cómo se puntúa y cómo se revisa

La usan **dos veces**: el equipo, para revisar su propio trabajo antes de entregar (es lo que dice la *Definition of Done*), y el docente, para puntuar. Se mira **cada criterio por separado** y se pone el nivel de la columna cuyo texto **se cumple entero**: si lo de un nivel se cumple a medias, es el nivel de abajo. **0** significa que no se ha entregado nada de ese criterio.

Orientación: lo que pide el nivel mínimo sale en un **2**, el estándar en un **3** y los extras en un **4**.

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Corrección técnica (¿funciona?)	Hay código, pero fallan muchas pruebas o no se ejecuta con el CSV real	Pasan las pruebas de lectura, cancelación e ingreso medio; sale `salida.json` con el CSV real; un error de fichero o de columna da mensaje y código 1	`python test_kpis.py` da **12 de 12**; la ocupación y la sección `calidad` salen bien; cada función tiene docstring y el estilo es PEP 8	Además, pruebas propias que pasan (lista vacía, división entre cero…) o `kpis_pandas.py` con los mismos resultados

Criterio	1 · Empezado	2 · Mínimo	3 · Estándar	4 · Extra
Reproducibilidad (¿lo puede repetir otra persona?)	Solo funciona en el ordenador de quien lo escribió	El README tiene los comandos, pero faltan datos (carpeta, versión de Python…)	Otra persona obtiene **el mismo JSON y las pruebas en verde** siguiendo solo el README; hay 3 o más *commits* de personas distintas, sin claves ni datos personales	Además, el README indica versión de Python, estructura de carpetas y el comando exacto
Análisis y comunicación al cliente (¿lo entiende quien no programa?)	El README no explica qué es cada KPI	Define los tres KPI, pero no sus límites	Define cada KPI **y explica por qué la ocupación es un proxy** (qué capacidad se supuso), sin jerga	Además, explica cómo cambia la ocupación si cambia la capacidad, o qué conclusión saca el cliente del resultado
IA responsable (¿se sabe qué hizo la IA y qué comprobasteis?)	No dice si se ha usado IA	Dice que se ha usado IA, sin detalle	Dice **qué se pidió, qué se revisó y qué se cambió**, y quien escribe sabe explicar cualquier línea	Además, cuenta un fallo concreto de la IA que detectaron las pruebas, y menciona que los datos son sintéticos y no hay datos personales

Cómo se revisa (unos 10 minutos, antes de subir; lo hace **una persona que no escribió esa parte**):

1. **Ejecutar**, en una carpeta limpia, `python test_kpis.py` y `python kpis.py reservas_turisdata.csv salida.json --capacidad 40`, copiando solo lo que dice el README. Anotar si algo no funciona.
2. **Leer el README como si fuera el cliente**: ¿se entiende qué hace cada KPI y por qué la ocupación es un proxy? Subrayar lo que no se entienda.
3. **Mirar el código**: ¿tiene docstring cada función? ¿Hay algo que no sepáis explicar? Que quien lo escribió lo explique.
4. **Poner un nivel (1 a 4) en cada criterio, con una frase de prueba** (por ejemplo: «3 · 12 de 12 pruebas, pero falta el docstring de `main`»). Sin prueba escrita, el nivel no vale.
5. **Apuntar el resultado** en la sección «Revisión con la rúbrica» del `README.md` y corregir lo más fácil de arreglar. Lo demás se anota como mejora.

Cada persona, además, se puntúa a sí misma en su hoja de autoevaluación y coevaluación con **esta misma rúbrica**.

4 · El README del equipo

Copiad `README_PLANTILLA.md` como `README.md` y rellenadlo (unos 10 minutos). Cada sección de la plantilla pide una cosa. Para el **nivel mínimo** bastan estas cuatro, en pocas líneas:

Sección de la plantilla	Qué escribís
Qué hace	Dos o tres frases: «`kpis.py` lee `reservas_turisdata.csv`, calcula tres KPI, avisa si el fichero está mal y guarda un JSON»
Cómo se ejecuta y cómo se prueba	Los comandos `python kpis.py reservas_turisdata.csv salida.json --capacidad 40` y `python test_kpis.py`, y el resultado (`12 de 12 pruebas superadas`)
KPIs y decisiones	Qué es cada KPI y **por qué la ocupación es un proxy**. Ejemplo: «No sabemos cuántas habitaciones tiene el hotel: suponemos 40 y, con 20, saldría el doble. Además contamos todas las noches el día de llegada»
Uso de IA y límites	Qué le pedisteis a la IA y qué revisasteis; los datos son sintéticos y no hay datos personales

Para el **nivel estándar**, completad también «Requisitos», «Gestión de errores» (qué errores controla, qué mensaje da y que devuelve código 1; por ejemplo: «Si falta el fichero, el programa muestra `ERROR: No se encuentra el fichero: …` y termina con código 1») y la versión con pandas si la hacéis.

La **revisión con la rúbrica** (apartado 3, unos 5 minutos) se anota en la sección «Revisión con la rúbrica» del final del README: un nivel de 1 a 4 por criterio y **una frase de prueba**. Si hay tiempo, arreglad lo más fácil; lo demás queda como «lo primero que mejoraríamos».

5 · ⬆ Subir los ficheros a GitHub

Unos 10 minutos. Hacedlo en **tres o más subidas, por personas distintas**: cada subida es un *commit* con el nombre de quien la hace.

- **Desde el navegador (la vía del curso).** Seguid Cómo entregar en GitHub: (1) abrid el repositorio del equipo y **entrad primero en la carpeta `sprint-04`**; (2) *Add file → Upload files*; (3) arrastrad los ficheros; (4) escribid un mensaje corto y pulsad **Commit changes**. Un reparto posible: la persona A sube `kpis.py` («Añado lectura del CSV y KPI»), la B sube `test_kpis.py` y `salida.json` («Añado pruebas y resultado») y la C sube `README.md` («Añado README»). Los nombres de archivo, **en minúsculas y sin espacios**. El CSV va a `data/` del repositorio solo si aún no está.
- **Con Git en el terminal** (si ya lo habéis practicado en `S04_05`): `git add kpis.py test_kpis.py README.md salida.json`, `git commit -m "Añado lectura del CSV y KPI"` y `git push`. Cada persona lo hace con su propia cuenta.
- **Cada persona sube además su hoja** de autoevaluación y coevaluación (`autoevaluacion_nombre.md`).

Antes de dar el sprint por cerrado, marcad la lista «Antes de cerrar el sprint» del README del sprint.